Practical – 1

1. **To study DDL and DML basic commands: [Annexure I]**

Q-1 Retrieve all employees' details.

A

```
SQL> SELECT * FROM employees
  2  ;

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        100 Steven               King
SKING                     515.123.4567         17-JUN-03 AD_PRES         24000
                                     90

        101 Neena                Kochhar
NKOCHHAR                  515.123.4568         21-SEP-05 AD_VP           17000
                      100            90

        102 Lex                  De Haan
LDEHAAN                   515.123.4569         13-JAN-01 AD_VP           17000
                      100            90

        103 Alexander            Hunold
AHUNOLD                   590.423.4567         03-JAN-06 IT_PROG          9000
                      102            60
```

Q-2 Write a query to display the last name, department number, and salary for all employees.

A select last_name,department_id,salary from employees;

```
LAST_NAME                 DEPARTMENT_ID     SALARY
------------------------- ------------- ----------
King                                 90      24000
Kochhar                              90      17000
De Haan                              90      17000
Hunold                               60       9000
Ernst                                60       6000
Austin                               60       4800
Pataballa                            60       4800
Lorentz                              60       4200
Greenberg                           100      12008
Faviet                              100       9000
Chen                                100       8200
Sciarra                             100       7700
Urman                               100       7800
Popp                                100       6900
```

Q-3 Find employees whose salary is between $5,000 and $10,000.

A

```
SQL> SELECT * FROM employees WHERE salary BETWEEN 5000 AND 10000;

EMPLOYEE_ID FIRST_NAME           LAST_NAME
---------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        103 Alexander            Hunold
AHUNOLD                   590.423.4567         03-JAN-06 IT_PROG          9000
                      102            60

        104 Bruce                Ernst
BERNST                    590.423.4568         21-MAY-07 IT_PROG          6000
                      103            60

        109 Daniel               Faviet
DFAVIET                   515.124.4169         16-AUG-02 FI_ACCOUNT       9000
                      108           100

        110 John                 Chen
JCHEN                     515.124.4269         28-SEP-05 FI_ACCOUNT       8200
                      108           100
```

Q-4 Retrieve employees working in specific departments (e.g., 50, 60).

```
SQL> SELECT * FROM employees WHERE department_id = 50;

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        120 Matthew              Weiss
MWEISS                    650.123.1234         18-JUL-04 ST_MAN           8000
                      100            50

        121 Adam                 Fripp
AFRIPP                    650.123.2234         10-APR-05 ST_MAN           8200
                      100            50

        122 Payam                Kaufling
PKAUFLIN                  650.123.3234         01-MAY-03 ST_MAN           7900
                      100            50

        123 Shanta               Vollman
SVOLLMAN                  650.123.4234         10-OCT-05 ST_MAN           6500
                      100            50
```

Q-5 List employees whose names start with the letter 'A'.

```
SQL> SELECT * FROM employees WHERE FIRST_NAME LIKE 'A%';

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        103 Alexander            Hunold
AHUNOLD                   590.423.4567         03-JAN-06 IT_PROG          9000
                      102            60

        115 Alexander            Khoo
AKHOO                     515.127.4562         18-MAY-03 PU_CLERK         3100
                      114            30

        121 Adam                 Fripp
AFRIPP                    650.123.2234         10-APR-05 ST_MAN           8200
                      100            50

        147 Alberto              Errazuriz
AERRAZUR                  011.44.1344.429278   10-MAR-05 SA_MAN          12000
            .3        100            80
```

Q-6	Find employees hired in 2022.

```
SQL> SELECT * FROM employees WHERE TO_CHAR(HIRE_DATE, 'YYYY') = '2002';

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        108 Nancy                Greenberg
NGREENBE                  515.124.4569         17-AUG-02 FI_MGR          12008
                      101           100

        109 Daniel               Faviet
DFAVIET                   515.124.4169         16-AUG-02 FI_ACCOUNT       9000
                      108           100

        114 Den                  Raphaely
DRAPHEAL                  515.127.4561         07-DEC-02 PU_MAN          11000
                      100            30

        203 Susan                Mavris
SMAVRIS                   515.123.7777         07-JUN-02 HR_REP           6500
                      101            40
```

Q-7	Show employees not working in departments 10 or 20.

```
SQL> SELECT * FROM employees WHERE department_id NOT IN (10, 20);

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        100 Steven               King
SKING                     515.123.4567         17-JUN-03 AD_PRES         24000
                                     90

        101 Neena                Kochhar
NKOCHHAR                  515.123.4568         21-SEP-05 AD_VP           17000
                      100            90

        102 Lex                  De Haan
LDEHAAN                   515.123.4569         13-JAN-01 AD_VP           17000
                      100            90

        103 Alexander            Hunold
AHUNOLD                   590.423.4567         03-JAN-06 IT_PROG          9000
                      102            60
```

Q-8 Retrieve employees who are not managers.

```
SQL> SELECT * FROM employees WHERE MANAGER_ID IS NULL;

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        100 Steven               King
SKING                     515.123.4567         17-JUN-03 AD_PRES         24000
                                     90
```

Q-9 List employees whose job title is 'Sales Manager' or 'Accountant'.

```
SQL> SELECT * FROM employees WHERE JOB_ID IN ('SA_MAN', 'AC_ACCOUNT');

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        145 John                 Russell
JRUSSEL                   011.44.1344.429268   01-OCT-04 SA_MAN          14000
            .4        100            80

        146 Karen                Partners
KPARTNER                  011.44.1344.467268   05-JAN-05 SA_MAN          13500
            .3        100            80

        147 Alberto              Errazuriz
AERRAZUR                  011.44.1344.429278   10-MAR-05 SA_MAN          12000
            .3        100            80

        148 Gerald               Cambrault
GCAMBRAU                  011.44.1344.619268   15-OCT-07 SA_MAN          11000
            .3        100            80
```

Q-10	Find employees with last names ending in 'son'.

```
SQL> SELECT * FROM employees WHERE LAST_NAME LIKE '%son';

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        130 Mozhe                Atkinson
MATKINSO                  650.124.6234         30-OCT-05 ST_CLERK         2800
                      121            50

        132 TJ                   Olson
TJOLSON                   650.124.8234         10-APR-07 ST_CLERK         2100
                      121            50

        179 Charles              Johnson
CJOHNSON                  011.44.1644.429262   04-JAN-08 SA_REP           6200
            .1        149            80
```

Q-11	Show employees who earn less than $4,000 or more than $15,000.

```
SQL> SELECT * FROM employees WHERE salary < 4000 OR salary > 15000;

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        100 Steven               King
SKING                     515.123.4567         17-JUN-03 AD_PRES         24000
                                     90

        101 Neena                Kochhar
NKOCHHAR                  515.123.4568         21-SEP-05 AD_VP           17000
                      100            90

        102 Lex                  De Haan
LDEHAAN                   515.123.4569         13-JAN-01 AD_VP           17000
                      100            90

        115 Alexander            Khoo
AKHOO                     515.127.4562         18-MAY-03 PU_CLERK         3100
                      114            30
```

Q-12	List employees hired before February 23, 2006.

```
SQL> SELECT *FROM employees WHERE HIRE_DATE < TO_DATE('23-FEB-2006', 'DD-MON-YYYY');

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        100 Steven               King
SKING                     515.123.4567         17-JUN-03 AD_PRES         24000
                                     90

        101 Neena                Kochhar
NKOCHHAR                  515.123.4568         21-SEP-05 AD_VP           17000
                      100            90

        102 Lex                  De Haan
LDEHAAN                   515.123.4569         13-JAN-01 AD_VP           17000
                      100            90

        103 Alexander            Hunold
AHUNOLD                   590.423.4567         03-JAN-06 IT_PROG          9000
                      102            60
```

Q-13	Find employees in department 90 earning more than $15,000.

```
SQL> SELECT *FROM employees WHERE DEPARTMENT_ID = 90  AND  SALARY > 15000;

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        100 Steven               King
SKING                     515.123.4567         17-JUN-03 AD_PRES         24000
                                     90

        101 Neena                Kochhar
NKOCHHAR                  515.123.4568         21-SEP-05 AD_VP           17000
                      100            90

        102 Lex                  De Haan
LDEHAAN                   515.123.4569         13-JAN-01 AD_VP           17000
                      100            90


3 rows selected.
```

Q-14	List employees whose names start with 'J' and who were hired after 2010.

```
SQL> SELECT *FROM employees WHERE FIRST_NAME LIKE 'J%'   AND TO_CHAR(HIRE_DATE, 'YYYY') > '2001';

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        110 John                 Chen
JCHEN                     515.124.4269         28-SEP-05 FI_ACCOUNT       8200
                      108           100

        112 Jose Manuel          Urman
JMURMAN                   515.124.4469         07-MAR-06 FI_ACCOUNT       7800
                      108           100

        125 Julia                Nayer
JNAYER                    650.124.1214         16-JUL-05 ST_CLERK         3200
                      120            50
```

Q-15	Retrieve employees not earning a commission.

```
SQL> SELECT *FROM employees WHERE commission_pct IS NULL OR commission_pct = 0;

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        100 Steven               King
SKING                     515.123.4567         17-JUN-03 AD_PRES         24000
                                         90

        101 Neena                Kochhar
NKOCHHAR                  515.123.4568         21-SEP-05 AD_VP           17000
                      100            90

        102 Lex                  De Haan
LDEHAAN                   515.123.4569         13-JAN-01 AD_VP           17000
                      100            90
```

Q-16	Find employees who are either in department 60 or have a manager with ID 101.

```
SQL> SELECT *FROM employees WHERE DEPARTMENT_ID = 60   OR MANAGER_ID=101;

EMPLOYEE_ID FIRST_NAME           LAST_NAME
----------- -------------------- -------------------------
EMAIL                     PHONE_NUMBER         HIRE_DATE JOB_ID         SALARY
------------------------- -------------------- --------- ---------- ----------
COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-------------- ---------- -------------
        103 Alexander            Hunold
AHUNOLD                   590.423.4567         03-JAN-06 IT_PROG          9000
                      102            60

        104 Bruce                Ernst
BERNST                    590.423.4568         21-MAY-07 IT_PROG          6000
                      103            60

        105 David                Austin
DAUSTIN                   590.423.4569         25-JUN-05 IT_PROG          4800
                      103            60
```

Q-2	List all courses offered in the Fall semester.

Annexure I: HR-Schema

```
SQL> desc employees
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 EMPLOYEE_ID                               NOT NULL NUMBER(6)
 FIRST_NAME                                         VARCHAR2(20)
 LAST_NAME                                 NOT NULL VARCHAR2(25)
 EMAIL                                     NOT NULL VARCHAR2(25)
 PHONE_NUMBER                                       VARCHAR2(20)
 HIRE_DATE                                 NOT NULL DATE
 JOB_ID                                    NOT NULL VARCHAR2(10)
 SALARY                                             NUMBER(8,2)
 COMMISSION_PCT                                     NUMBER(2,2)
 MANAGER_ID                                         NUMBER(6)
 DEPARTMENT_ID                                      NUMBER(4)
```

Practical – 2[Additional]

2. **Perform following queries**

Q-1 Retrieve all data from **employee, jobs and deposit.**

A

```
SQL> select * from employee;

    EMP_NO EMP_NAME                          EMP_SAL   EMP_COMM    DEPT_NO L_NAME                         DEPT_NAME                      JOB_ID          LOCATION        MANAGER_ID HIREDATE
---------- ------------------------------ ---------- ---------- ---------- ------------------------------ ------------------------------ --------------- --------------- ---------- ---------
       101 Smith                                 800          0         20 shah                           machine learning               fi_mgr          toronto                105 09-AUG-96
       107 Anamika                              2975          0         30 jha                            artificial intelligence        it_prog         new york               105 15-JUL-97
       106 Sneha                                2450      24500         10 joseph                         big data analytics             fi_acc          melbourne              105 26-SEP-97
       103 Adama                                1100          0         20 wales                          machine learning               mk_mgr          ontario                105 30-NOV-95
       102 Snehal                               1600        300         25 gupta                          data science                   lec             las vegas              105 14-MAR-96
```

```
SQL> select * from job;

JOB_ID          JOB_TITLE                          MIN_SAL    MAX_SAL
--------------- ------------------------------ ---------- ----------
lec             Lecturer                             6000      17000
it_prog         Programmer                           4000      10000
mk_mgr          Marketing manager                    9000      15000
fi_mgr          Finance manager                      8200      12000
fi_acc          Account                              4200       9000
```

```
SQL> select * from deposit
  2  ;

A_NO  CNAME           BNAME               AMOUNT A_DATE
----- --------------- --------------- ---------- ---------
102   Sunil           virar                 5000 15-JUL-06
103   Jay             villeparle            6500 12-MAR-06
101   Anil            andheri               7000 01-JAN-06
104   Vijay           andheri               8000 17-SEP-06
105   Keyur           Dadar                 7500 19-NOV-06
106   Mayur           borivali              5500 21-DEC-06

6 rows selected.
```

Q-2 Give details of account no. and deposited rupees of customers having account opened between dates **01-01-06 and 25-07-06**.

A

```
SQL> SELECT a_no, amount
  2  FROM Deposit
  3  WHERE a_date BETWEEN TO_DATE('01-JAN-06', 'DD-MON-YY')
  4                   AND TO_DATE('25-JUL-06', 'DD-MON-YY');

A_NO      AMOUNT
----- ----------
102         5000
103         6500
101         7000
```

Q-3 Display all jobs with minimum salary is greater than 4000.

A

```
SQL> SELECT *
  2  FROM Job
  3  WHERE min_sal > 4000;

JOB_ID          JOB_TITLE                         MIN_SAL    MAX_SAL
--------------- ------------------------------ ---------- ----------
lec             Lecturer                             6000      17000
mk_mgr          Marketing manager                    9000      15000
fi_mgr          Finance manager                      8200      12000
fi_acc          Account                              4200       9000
```

Q-4 Display name and salary of employee whose department no is 20. Give alias name to name of employee.

A

```
SQL> SELECT emp_name AS employee_name, emp_sal
  2  FROM Employee
  3  WHERE dept_no = 20;

EMPLOYEE_NAME                     EMP_SAL
------------------------------ ----------
Smith                                 800
Adama                                1100
```

Q-5 Display employee no, name and department details of those employee whose department lies **in (10,20).**

A

```
SQL> SELECT emp_no, emp_name, dept_no
  2  FROM Employee
  3  WHERE dept_no IN (10, 20);

    EMP_NO EMP_NAME                          DEPT_NO
---------- ------------------------------ ----------
       101 Smith                                  20
       106 Sneha                                  10
       103 Adama                                  20
```

Q-6 Display the **non-null** values of employees.

A

```
SQL> SELECT *
  2  FROM Employee
  3  WHERE emp_name IS NOT NULL AND emp_sal IS NOT NULL AND dept_no IS NOT NULL;

    EMP_NO EMP_NAME                          EMP_SAL   EMP_COMM    DEPT_NO L_NAME                         DEPT_NAME                      JOB_ID          LOCATION        MANAGER_ID HIREDATE
---------- ------------------------------ ---------- ---------- ---------- ------------------------------ ------------------------------ --------------- --------------- ---------- ---------
       101 Smith                                 800          0         20 shah                           machine learning               fi_mgr          toronto                105 09-AUG-96
       107 Anamika                              2975          0         30 jha                            artificial intelligence        it_prog         new york               105 15-JUL-97
       106 Sneha                                2450      24500         10 joseph                         big data analytics             fi_acc          melbourne              105 26-SEP-97
       103 Adama                                1100          0         20 wales                          machine learning               mk_mgr          ontario                105 30-NOV-95
       102 Snehal                               1600        300         25 gupta                          data science                   lec             las vegas              105 14-MAR-96
```

Q-7 Display name of customer along with its account no **(both columns should be displayed as one)** whose amount is not equal to 8000 Rs.

A

```
SQL> select cid || ' ' || a_no AS "Customer Account" from deposit where amount != 8000;

Customer Account
--------------------------------------------
1 101
2 102
3 103
5 105
6 106
```

Q-8 Display the content of job details with minimum salary **either 2000 or 4000**.

A

```
SQL> SELECT *
  2  FROM Job
  3  WHERE min_sal IN (2000, 4000);

JOB_ID          JOB_TITLE                         MIN_SAL    MAX_SAL
--------------- ------------------------------ ---------- ----------
it_prog         Programmer                           4000      10000
```

Q. **To study various options of LIKE predicate**

Q-1 Display all employee whose name start with ‘A’ and third character is ‘‘a’.

A

```
SQL> select * from employee where emp_name like 'A_a%';

    EMP_NO EMP_NAME                          EMP_SAL   EMP_COMM    DEPT_NO L_NAME                         DEPT_NAME                      JOB_ID          LOCATION        MANAGER_ID HIREDATE
---------- ------------------------------ ---------- ---------- ---------- ------------------------------ ------------------------------ --------------- --------------- ---------- ---------
       103 Adama                                1100          0         20 Wales                          Machine Learning               mk_mgr          Ontario                105 30-NOV-95
       104 Aman                                 3000         15         12 Sharma                         Virtual Reality                comp_op         Mexico                  12 02-OCT-97
       107 Anamika                              2975         30         15 Jha                            Artificial Intelligence        it_prog         New York                15 15-JUL-97
```

Q-2 Display name, number and salary of those employees whose name is 5 characters long and first three characters are ‘Ani’.

A

```
SQL> select emp_name, emp_no, emp_sal from employee where emp_name like 'Ani__';

EMP_NAME                           EMP_NO    EMP_SAL
------------------------------ ---------- ----------
Anita                                 105       5000
```

Q-3 Display all information of employee whose second character of name is either ‘M’ or ‘N’.

A

```
SQL> select * from employee where emp_name like '_m%' or emp_name like '_n%';

    EMP_NO EMP_NAME                       EMP_SAL   EMP_COMM    DEPT_NO L_NAME                         DEPT_NAME                      JOB_ID          LOCATION        MANAGER_ID HIREDATE
---------- ------------------------------ ---------- ---------- ---------- ------------------------------ ------------------------------ --------------- --------------- ---------- ---------
       101 Smith                                 800         20          1 Shah                           Machine Learning               fi_mgr          Toronto                105 09-AUG-96
       102 Snehal                               1600        300         25 Gupta                          Data Science                   lec             Las Vegas               14 14-MAR-96
       104 Aman                                 3000         15         12 Sharma                         Virtual Reality                comp_op         Mexico                  12 02-OCT-97
       105 Anita                                5000      50000         10 Patel                          Big Data Analytics             comp_op         Germany                107 01-JAN-98
       106 Sneha                                2450      24500         10 Joseph                         Big Data Analytics             fi_acc          Melbourne              105 26-SEP-97
       107 Anamika                              2975         30         15 Jha                            Artificial Intelligence        it_prog         New York                15 15-JUL-97

6 rows selected.
```

Q-4 Find the list of all customer name whose branch is in ‘andheri’ or ‘dadar’ or ‘virar’.

A

```
SQL> select cid from deposit where bname in ('Andheri', 'Dadar','Virar');

       CID
----------
         1
         2
         4
         5
```

Q-5 Display the job name whose first three character in job id field is ‘FI_’.

A

```
SQL> select job_title from job where job_id like 'fi\_%' escape '\';

JOB_TITLE
------------------------------
Account
Finance Manager
```

Q-6 Display the title/name of job who’s last three character are ‘**_MGR**’ and their maximum salary is greater than **Rs 12000**.

A

```
SQL> select job_title from job where job_id like '%\_mgr' escape '\' and max_sal > 12000;

JOB_TITLE
------------------------------
Marketing Manager
```

Q-7 Display the non-null values of employees and also employee name second character should be ‘n’ and string should be 5-character long.

A

```
SQL> select * from employee where emp_name is not null and emp_name like '_n___';

    EMP_NO EMP_NAME                       EMP_SAL   EMP_COMM    DEPT_NO L_NAME                         DEPT_NAME                      JOB_ID          LOCATION        MANAGER_ID HIREDATE
---------- ------------------------------ ---------- ---------- ---------- ------------------------------ ------------------------------ --------------- --------------- ---------- ---------
       105 Anita                                5000      50000         10 Patel                          Big Data Analytics             comp_op         Germany                107 01-JAN-98
       106 Sneha                                2450      24500         10 Joseph                         Big Data Analytics             fi_acc          Melbourne              105 26-SEP-97
```

Q-8 Display the null values of employee and also employee name's third character should be 'a'.

A

```
SQL> select * from employee where emp_name is null or emp_name like '__a%';

    EMP_NO EMP_NAME                       EMP_SAL   EMP_COMM    DEPT_NO L_NAME                         DEPT_NAME                      JOB_ID          LOCATION        MANAGER_ID HIREDATE
---------- ------------------------------ ---------- ---------- ---------- ------------------------------ ------------------------------ --------------- --------------- ---------- ---------
       103 Adama                                1100          0         20 Wales                          Machine Learning               mk_mgr          Ontario                105 30-NOV-95
       104 Aman                                 3000         15         12 Sharma                         Virtual Reality                comp_op         Mexico                  12 02-OCT-97
       107 Anamika                              2975         30         15 Jha                            Artificial Intelligence        it_prog         New York                15 15-JUL-97
```

Q-9 What will be output if you are giving LIKE predicate as '%_%' ESCAPE '\'

A

```
SQL> select * from job where job_id like '%\_%' escape '\';

JOB_ID          JOB_TITLE                         MIN_SAL    MAX_SAL
--------------- ------------------------------ ---------- ----------
it_prog         Programmer                           4000      10000
mk_mgr          Marketing Manager                    9000      15000
fi_mgr          Finance Manager                      8200      12000
fi_acc          Account                              4200       9000
comp_op         Computer Operator                    1500       3000
```

Annexure II: Practice Schema

- Create Query of all Tables.

```
SQL> CREATE TABLE Job (
  2      job_id VARCHAR2(15) PRIMARY KEY,
  3      job_title VARCHAR2(30) NOT NULL,
  4      min_sal NUMBER(7,2) CHECK (min_sal >= 0),
  5      max_sal NUMBER(7,2),
  6      CONSTRAINT chk_salary_range CHECK (max_sal >= min_sal)
  7  );

Table created.
```

```
SQL> CREATE TABLE Employee (
        emp_no NUMBER(3) PRIMARY KEY,
        emp_name VARCHAR2(30) NOT NULL,
        emp_sal NUMBER(8,2) CHECK (emp_sal > 0),
        emp_comm NUMBER(6,1) DEFAULT 0 CHECK (emp_comm >= 0),
        dept_no NUMBER(3) NOT NULL,
        l_name VARCHAR2(30),
        dept_name VARCHAR2(30),
        job_id VARCHAR2(15) REFERENCES Job(job_id) ON DELETE SET NULL,
        location VARCHAR2(15),
        manager_id NUMBER(5),
        hiredate DATE NOT NULL
    );

Table created.
```

```
SQL> CREATE TABLE Branch (
        bname VARCHAR2(50) PRIMARY KEY,
        city VARCHAR2(50) NOT NULL
    );

Table created.
```

```
SQL> CREATE TABLE Customer (
        cid NUMBER(10) PRIMARY KEY,
        cname VARCHAR2(50) NOT NULL,
        city VARCHAR2(50) NOT NULL
    );

Table created.
```

```
SQL> CREATE TABLE Deposit (
        a_no VARCHAR2(5) PRIMARY KEY,
        cid NUMBER(10) NOT NULL,
        bname VARCHAR2(10) NOT NULL,
        amount NUMBER(7,2),
        a_date DATE,
        CONSTRAINT fk_cid FOREIGN KEY (cid) REFERENCES Customer(cid),
        CONSTRAINT fk_bname FOREIGN KEY (bname) REFERENCES Branch(bname)
    );

Table created.
```

```
SQL> CREATE TABLE borrow (
  2      loanno VARCHAR2(5) PRIMARY KEY,
  3      cid NUMBER(10) NOT NULL,
  4      bname VARCHAR2(10) NOT NULL,
  5      amount NUMBER(7,2),
  6      CONSTRAINT fk_borrow_cid FOREIGN KEY (cid) REFERENCES Customer(cid),
  7      CONSTRAINT fk_borrow_bname FOREIGN KEY (bname) REFERENCES Branch(bname)
  8  );

Table created.
```

Practical – 2

1. **Perform the following queries: [Annexure II]**

Q-1 Find all instructors in the Computer Science department:

A

```
SQL> select * from instructor where dept_name='Comp. Sci.';

ID    NAME                 DEPT_NAME                SALARY
----- -------------------- -------------------- ----------
34175 Bondi                Comp. Sci.            115469.11
3335  Bourrier             Comp. Sci.             80797.83
```

Q-2 List all courses offered in the Fall semester:

A

```
SQL> SELECT * FROM section WHERE semester = 'Fall';

COURSE_I SEC_ID   SEMEST       YEAR BUILDING        ROOM_NU TIME
-------- -------- ------ ---------- --------------- ------- ----
313      1        Fall         2010 Chandler        804     N
893      1        Fall         2007 Fairchild       145     B
559      1        Fall         2002 Lamberton       134     J
239      1        Fall         2006 Taylor          183     C
158      1        Fall         2008 Whitman         434     F
376      1        Fall         2006 Power           717     K
927      1        Fall         2002 Saucon          180     F
```

Q-3 Find students who have taken courses between a range of 100 to 250 course IDs:

A

```
SQL>
SQL> SELECT DISTINCT student.name
  2  FROM student
  3  JOIN takes ON student.ID = takes.ID
  4  WHERE TO_NUMBER(course_id) BETWEEN 100 AND 250;

NAME
--------------------
Nives
Quimby
Rzecz
Haigh
Teo
Cochran
```

Q-4 Get all instructors teaching a specific course and whose salaries are within min of 30000 and 90000.

A

```
SQL> SELECT instructor.* FROM instructor JOIN teaches ON instructor.ID = teaches.ID WHERE teaches.course_id = '345' AND instructor.salary BETWEEN 30000 AND 90000;

ID    NAME                 DEPT_NAME                SALARY
----- -------------------- -------------------- ----------
79081 Ullman               Accounting              47307.1
```

Q-5 Get details of all sections where a particular classroom is used and the section is scheduled between specific times:

A

```
SQL>
SQL> SELECT *
  2  FROM section
  3  WHERE building = 'Fairchild'
  4    AND room_number = '145'
  5    AND time_slot_id BETWEEN 'B' AND 'D';

COURSE_I SEC_ID   SEMEST       YEAR BUILDING        ROOM_NU TIME
-------- -------- ------ ---------- --------------- ------- ----
893      1        Fall         2007 Fairchild       145     B
663      1        Spring       2005 Fairchild       145     D
242      1        Fall         2009 Fairchild       145     C
```

Q-6 Get all instructors teaching courses in English department and whose ID between 20000 to 70000.

A

```
SQL> SELECT DISTINCT i.ID, i.name
  2  FROM instructor i
  3  JOIN teaches t ON i.ID = t.ID
  4  JOIN course c ON t.course_id = c.course_id
  5  WHERE c.dept_name = 'English' AND i.ID BETWEEN '20000' AND '70000';

ID    NAME
----- --------------------
28097 Kean
```

Q-7 Display name of instructor along with its ID (both columns should be displayed as one).

A

```
SQL> SELECT name || ' (' || ID || ')' AS InstructorDetails
  2  FROM instructor;

INSTRUCTORDETAILS
----------------------------
McKinnon (63395)
Pingr (78699)
Mird (96895)
Luo (4233)
Murata (4034)
Konstantinides (50885)
Levine (79653)
Shuming (50330)
Queiroz (80759)
Sullivan (73623)
Bertolino (97302)
```

Q-8 Retrieve courses whose credits fall between 3 and 5 and display with new alias name.

A

```
SQL> SELECT course_id AS CourseID, title AS CourseName
  2  FROM course
  3  WHERE credits BETWEEN 3 AND 5;

COURSEID COURSENAME
-------- --------------------------------------------------
787      C  Programming
238      The Music of Donovan
608      Electron Microscopy
539      International Finance
278      Greek Tragedy
972      Greek Tragedy
391      Virology
814      Compiler Design
272      Geology
612      Mobile Computing
237      Surfing
```

Q-9 Find students who have earned between 10 and 20 total credits.

A

```
SQL> SELECT *
  2  FROM student
  3  WHERE tot_cred BETWEEN 10 AND 20;

ID    NAME                 DEPT_NAME              TOT_CRED
----- -------------------- -------------------- ----------
65714 Hughes               English                      19
41890 Srivastava           Physics                      15
93508 Graham               Physics                      14
30845 Fonseca              Math                         19
2561  Aschoff              Finance                      20
64731 Yuanq                Languages                    13
90009 Donofrio             Pol. Sci.                    13
55009 Pohlem               Pol. Sci.                    18
75791 Keuk                 Finance                      11
41491 Beavis               Comp. Sci.                   15
50537 Felling              Mech. Eng.                   20
```

Q-10 List courses offered by specific departments (e.g., 'CS', 'Physics', 'Math').

A

```
SQL> SELECT *
  2  FROM course
  3  WHERE dept_name IN ('CS', 'Physics', 'Math');

COURSE_I TITLE                                              DEPT_NAME            CREDITS
-------- -------------------------------------------------- -------------------- ----------
612      Mobile Computing                                   Physics                       3
843      Environmental Law                                  Math                          4
679      The Beatles                                        Math                          3
376      Cost Accounting                                    Physics                       4
461      Physical Chemistry                                 Math                          3
959      Bacteriology                                       Physics                       4
235      International Trade                                Math                          3
```

Q-11 List all prerequisites that are not null for any course.

A

```
SQL> SELECT *
  2  FROM prereq
  3  WHERE prereq_id IS NOT NULL;

COURSE_I PREREQ_I
-------- --------
133      852
158      408
169      603
209      780
224      227
236      984
239      628
241      486
242      304
242      594
254      877
```

Q-12 Find all courses offered between two specific years 2000 and 2006.

A	<pre>SQL> SELECT DISTINCT course_id, title, dept_name, credits 2 FROM course 3 WHERE course_id IN (4 SELECT course_id 5 FROM section 6 WHERE year BETWEEN 2000 AND 2006 7); COURSE_I TITLE DEPT_NAME CREDITS -------- -- -------------------- ---------- 338 Graph Theory Psychology 3 400 Visual BASIC Psychology 4 760 How to Groom your Cat Accounting 3 629 Finite Element Analysis Cybernetics 3 482 FOCAL Programming Psychology 4 581 Calculus Pol. Sci. 4 591 Shakespeare Pol. Sci. 4 319 World History Finance 4 960 Tort Law Civil Eng. 3 274 Corporate Law Comp. Sci. 4 426 Video Gaming Finance 3</pre>
Q-13	Delete classrooms with a capacity less than 10.
A	<pre>SQL> DELETE FROM classroom WHERE CAPACITY<11; 5 rows deleted.</pre>
Q-14	Remove all sections offered before 2019.
A	<pre>SQL> DELETE FROM section WHERE year<2019; 100 rows deleted.</pre>
Q-15	Assign the department Information Technology to courses previously under Computer Science.
A	
Q-16	Give a 10% salary increase to instructors in the Physics department.
A	<pre>SQL> UPDATE instructor 2 SET salary = salary * 1.10 3 WHERE dept_name = 'Physics'; 2 rows updated.</pre>

Q-17	Update all instructors in the Mathematics department, giving them a 5% raise and changing their department to Applied Mathematics.
A	``SQL> UPDATE instructor`` ``2 SET salary = salary * 1.05,`` ``3 dept_name = 'Applied Mathematics'`` ``4 WHERE dept_name = 'Mathematics';`` ``0 rows updated.``

Aim: To study various options of LIKE predicate

Q-1	Find instructors whose names start with the letter 'A.'
A	SQL> SELECT * FROM instructor WHERE name LIKE 'A%'; ID NAME DEPT_NAME SALARY ----- -------------------- -------------------- ---------- 37687 Arias Statistics 104563.38 95030 Arinb Statistics 54805.11 28400 Atanassov Statistics 84982.92
Q-2	Find classrooms with a building name ending in "Hall."
A	SQL> SELECT * FROM classroom WHERE BUILDING LIKE '%HALL'; no rows selected
Q-3	Find courses with names containing the word "Data."
A	SQL> SELECT * FROM course WHERE TITLE LIKE '%DATA%'; no rows selected

Q-4	Find students whose names have 'a' as the second character.
A	See output below

```
SQL> SELECT * FROM student WHERE name LIKE '_a%';

ID    NAME                 DEPT_NAME              TOT_CRED
----- -------------------- -------------------- ----------
37339 Warren               Psychology                   68
15883 Marques              Math                         24
75547 Varadaran            Pol. Sci.                    96
61332 Canon                Astronomy                     8
68779 Harmon               Athletics                    47
51008 Kandadai             History                     107
31079 Canas                Astronomy                    85
9084  Rabu                 Finance                      87
94311 Napoletani           English                      40
50702 Harders              Math                         63
84808 Randers              Psychology                  104
```

Q-5	List departments whose names have a vowel as the second character.
A	See output below

```
SQL> SELECT *
  2  FROM department  WHERE dept_name LIKE '_a%'
  3     OR dept_name LIKE '_e%'
  4     OR dept_name LIKE '_i%'
  5     OR dept_name LIKE '_o%'
  6     OR dept_name LIKE '_u%';

DEPT_NAME            BUILDING            BUDGET
-------------------- --------------- ----------
Civil Eng.           Chandler         255041.46
Biology              Candlestick      647610.55
History              Taylor           699140.86
Marketing            Lambeau          210627.58
Pol. Sci.            Whitman          573745.09
Comp. Sci.           Lamberton        106378.69
Languages            Linderman         601283.6
Finance              Candlestick      866831.75
Geology              Palmer           406557.93
Math                 Brodhead         777605.11
Mech. Eng.           Rauch            520350.65

11 rows selected.
```

Q-6	Find all departments where the department name contains an underscore (_).
A	<pre>SQL> SELECT * FROM department WHERE dept_name LIKE ' %_% ' ; no rows selected</pre>
Q-7	Find courses with spaces in their titles.
A	<pre>SQL> SELECT * FROM course WHERE title LIKE '% %'; COURSE_I TITLE DEPT_NAME -------- -- -------------------- CREDITS ---------- 787 C Programming Mech. Eng. 4 394 C Programming Athletics 3 396 C Programming Languages 3</pre>

Annexure 2: HR-Schema

- List of tables in University-Schema.

```
SQL> SELECT table_name FROM user_tables;

TABLE_NAME
--------------------------------------------------------------------------------
REGIONS
COUNTRIES
LOCATIONS
DEPARTMENTS
JOBS
EMPLOYEES
JOB_HISTORY
JOB
EMPLOYEE
DEPOSIT
BRANCH

TABLE_NAME
--------------------------------------------------------------------------------
BORROW
CUSTOMER
CLASSROOM
DEPARTMENT
COURSE
INSTRUCTOR
SECTION
TEACHES
STUDENT
TAKES
ADVISOR

TABLE_NAME
--------------------------------------------------------------------------------
TIME_SLOT
PREREQ
```

- Insert data in above tables.

Table	Insert query:
advisor	insert into advisor values ('70807', '72553'), ('4355', '14365'), ('8457', '43779'), ('10904', '48570'), ('83836', '63287');
classroom	insert into classroom values ('Lamberton', 134, 10), ('Chandler', 375, 10), ('Fairchild', 145, 27), ('Nassau', 45, 92), ('Grace', 40, 34);
course	insert into course values ('787', 'C Programming', 'Mech. Eng.', 4), ('238', 'The Music of Donovan', 'Mech. Eng.', 3), ('608', 'Electron Microscopy', 'Mech. Eng.', 3), ('539', 'International Finance', 'Comp. Sci.', 3), ('278', 'Greek Tragedy', 'Statistics', 4);

Department	insert into department values ('Civil Eng.', 'Chandler', 255041.46), ('Biology', 'Candlestick', 647610.55), ('History', 'Taylor', 699140.86), ('Physics', 'Wrigley', 942162.76), ('Marketing', 'Lambeau', 210627.58);
instructor	insert into instructor values ('63395', 'McKinnon', 'Cybernetics', 94333.99), ('78699', 'Pingr', 'Statistics', 59303.62), ('96895', 'Mird', 'Marketing', 119921.41), ('4233', 'Luo', 'English', 88791.45), ('4034', 'Murata', 'Athletics', 61387.56);
prereq	insert into prereq values ('411', '401'), ('830', '748'), ('558', '130'), ('877', '599'), ('349', '612');

Section	insert into section values ('313', '1', 'Fall', 2010, 'Chandler', '804', 'N'), ('747', '1', 'Spring', 2004, 'Gates', '314', 'K'), ('443', '1', 'Spring', 2010, 'Whitman', '434', 'O'), ('893', '1', 'Fall', 2007, 'Fairchild', '145', 'B'), ('663', '1', 'Spring', 2005, 'Fairchild', '145', 'D');
student	insert into student values ('92839', 'Cirsto', 'Math', 115), ('79329', 'Velikovs', 'Marketing', 110), ('97101', 'Marek', 'Psychology', 53), ('24865', 'Tran-', 'Marketing', 116), ('36052', 'Guerra', 'Elec. Eng.', 59);
takes	insert into takes values ('31560', '679', '1', 'Spring', 2010, 'C-'), ('41261', '362', '2', 'Fall', 2006, 'A+'), ('19848', '258', '1', 'Fall', 2007, 'B '), ('84189', '105', '1', 'Fall', 2009, 'C+'), ('10727', '338', '1', 'Spring', 2007, 'C-');

Teaches	insert into teaches values ('77346', '486', '1', 'Fall', 2009), ('79081', '408', '1', 'Spring', 2007), ('6569', '362', '3', 'Spring', 2008), ('6569', '527', '1', 'Fall', 2004), ('41930', '401', '1', 'Fall', 2003);
time_slot	insert into time_slot values ('A', 'M', 8, 0, 8, 50), ('A', 'W', 8, 0, 8, 50), ('A', 'F', 8, 0, 8, 50), ('B', 'M', 9, 0, 9, 50), ('B', 'W', 9, 0, 9, 50);

	Practical – 3
3.	**To Perform various data manipulation commands, aggregate functions and sorting concept on all created tables.**
Q-1	Update the budget of the Computer Science department.
A	`SQL> UPDATE department SET budget = 500000 WHERE dept_name = 'Comp. Sci.';` `1 row updated.`
Q-2	Delete all sections offered in the year 2002.
A	`SQL> DELETE FROM section WHERE year = 2002;` `13 rows deleted.`
Q-3	Count the total number of courses offered.
A	`SQL> SELECT COUNT(*) AS total_courses FROM course;` `TOTAL_COURSES` `-------------` `          200`
Q-4	Find the total credits earned by all students
A	`SQL> SELECT SUM(tot_cred) AS total_credits FROM student;` `TOTAL_CREDITS` `-------------` `       132806`
Q-5	Calculate the average salary of instructors in the Physics department.
A	

```
SQL> select AVG(salary) AS avg_salary
  2  from instructor
  3  where dept_name = 'Physics';

AVG_SALARY
----------
 126034.59
```

Q-6 Find the minimum and maximum capacity of classroom.

A

```
SQL> SELECT MIN(capacity) AS min_capacity, MAX(capacity) AS max_capacity FROM classroom;

MIN_CAPACITY MAX_CAPACITY
------------ ------------
          10          120
```

Q-7 Get the number of students in each department.

A

```
SQL> SELECT dept_name, COUNT(*) AS num_students FROM student GROUP BY dept_name;

DEPT_NAME            NUM_STUDENTS
-------------------- ------------
Mech. Eng.                    105
Astronomy                     106
English                        95
Psychology                    100
Elec. Eng.                     98
Biology                       100
Cybernetics                    86
Math                           91
Physics                        96
Pol. Sci.                     109
Statistics                     85

DEPT_NAME            NUM_STUDENTS
-------------------- ------------
Languages                     119
Athletics                      92
Geology                        92
Finance                        97
History                       117
Civil Eng.                    120
Marketing                      85
Comp. Sci.                    108
Accounting                     99

20 rows selected.
```

Q-8 List departments with more than 10 students.

<table>
<tr><td>A</td><td>

```
SQL> SELECT dept_name FROM student GROUP BY dept_name HAVING COUNT(*) > 10;

DEPT_NAME
--------------------
Mech. Eng.
Astronomy
English
Psychology
Elec. Eng.
Biology
Cybernetics
Math
Physics
Pol. Sci.
Statistics

DEPT_NAME
--------------------
Languages
Athletics
Geology
Finance
History
Civil Eng.
Marketing
Comp. Sci.
Accounting

20 rows selected.
```

</td></tr>
<tr><td>Q-9</td><td>Sort instructors by their salary in descending order.</td></tr>
<tr><td>A</td><td>

```
SQL> SELECT * FROM instructor ORDER BY salary DESC;

ID    NAME                 DEPT_NAME                SALARY
----- -------------------- -------------------- ----------
19368 Wieland              Pol. Sci.             124651.41
74420 Voronina             Physics               121141.99
96895 Mird                 Marketing             119921.41
95709 Sakurai              English               118143.98
90376 Bietzk               Cybernetics            117836.5
34175 Bondi                Comp. Sci.            115469.11
50330 Shuming              Physics               108011.81
48507 Lent                 Mech. Eng.            107978.47
74426 Kenje                Marketing             106554.73
6569  Mingoz               Finance               105311.38
37687 Arias                Statistics            104563.38
```

</td></tr>
</table>

Q-10 Sort courses by department name (ascending) and credits (descending).

A

```
SQL> SELECT * FROM course ORDER BY dept_name ASC, credits DESC;

COURSE_I TITLE                                                  DEPT_NAME
-------- -------------------------------------------------- --------------------
   CREDITS
----------
598      Number Theory                                          Accounting
         4

200      The Music of the Ramones                               Accounting
         4

345      Race Car Driving                                       Accounting
         4


COURSE_I TITLE                                                  DEPT_NAME
-------- -------------------------------------------------- --------------------
   CREDITS
----------
344      Quantum Mechanics                                      Accounting
         4

408      Bankruptcy                                             Accounting
         3

760      How to Groom your Cat                                  Accounting
         3
```

Q-11 Display students sorted by their total credits in descending order.

A

```
SQL> SELECT * FROM student ORDER BY tot_cred DESC;

ID    NAME                 DEPT_NAME              TOT_CRED
----- -------------------- -------------------- ----------
82301 Conti                Marketing                   129
20803 Mercurio             History                     129
75560 Tabor                History                     129
14581 Vagn                 Biology                     129
72657 Hird                 Comp. Sci.                  129
33645 Kawakami             Comp. Sci.                  129
26427 Ende                 Finance                     129
93004 Gibbs                Finance                     129
38476 Rzecz                Pol. Sci.                   129
14214 Yoneda               Cybernetics                 129
71025 Cadis                History                     129
```

<table>
<tr><td>Q-12</td><td>Find the average salary of instructors in each department and sort by average salary in descending order.</td></tr>
<tr><td>A</td><td>

```
SQL> SELECT dept_name, AVG(salary) AS avg_salary FROM instructor GROUP BY dept_name ORDER BY avg_salary DESC;

DEPT_NAME            AVG_SALARY
-------------------- ----------
Physics                114576.9
Finance               105311.38
Pol. Sci.            100053.073
Geology                99382.59
Comp. Sci.             98133.47
Cybernetics          96346.5675
Marketing            84097.4375
Mech. Eng.             79813.02
Astronomy              79070.08
Athletics             77098.198
Elec. Eng.             74162.74

DEPT_NAME            AVG_SALARY
-------------------- ----------
English                72089.05
Statistics           67795.4417
Biology                61287.25
Psychology             61143.05
Languages            57421.8567
Accounting           48716.5925

17 rows selected.
```

</td></tr>
<tr><td>Q-13</td><td>List the top 3 departments with the highest budgets.</td></tr>
<tr><td>A</td><td>

```
SQL> SELECT dept_name
  2  FROM department
  3  ORDER BY budget DESC
  4  FETCH FIRST 3 ROWS ONLY;

DEPT_NAME
--------------------
Physics
Finance
Psychology
```

</td></tr>
</table>

<table>
<tr><td></td><td>Practical – 4</td></tr>
<tr><td>4.</td><td>To Implement Single-row functions.</td></tr>
<tr><td>Q-1</td><td>Write a query to display the current date. Label the column Date</td></tr>
<tr><td>A</td><td><pre>
SQL> select sysdate as column_Date from DUAL;

COLUMN_DA

05-FEB-25
</pre></td></tr>
<tr><td>Q-2</td><td>For each employee, display the employee number, salary, and salary increased by 15% and expressed as a whole number. Label the column New Salary</td></tr>
<tr><td>A</td><td><pre>
SQL> SELECT EMPLOYEE_ID,
  2   SALARY,
  3   ROUND(SALARY * 1.15) AS "New Salary"
  4 FROM employees;

EMPLOYEE_ID   SALARY New Salary
----------- ---------- ----------
        100   24000   27600
        101   17000   19550
        102   17000   19550
        103   9000   10350
        104   6000   6900
        105   4800   5520
        106   4800   5520
        107   4200   4830
        108   12008   13809
        109   9000   10350
        110   8200   9430
</pre></td></tr>
<tr><td>Q-3</td><td>Modify your query no (2) to add a column that subtracts the old salary from the new salary. Label the column Increase</td></tr>
</table>

A

```
SQL> SELECT EMPLOYEE_ID,
  2         SALARY,
  3         ROUND(SALARY * 1.15) AS "New Salary",
  4         ROUND(SALARY * 1.15) - SALARY AS "Increase"
  5  FROM employees;

EMPLOYEE_ID     SALARY New Salary   Increase
----------- ---------- ---------- ----------
        100      24000      27600       3600
        101      17000      19550       2550
        102      17000      19550       2550
        103       9000      10350       1350
        104       6000       6900        900
        105       4800       5520        720
        106       4800       5520        720
        107       4200       4830        630
        108      12008      13809       1801
        109       9000      10350       1350
        110       8200       9430       1230
```

Q-4

Write a query that displays the employee's names with the first letter capitalized and all other letters lowercase, and the length of the names, for all employees whose name starts with J, A, or M. Give each column an appropriate label. Sort the results by the employees' last names.

A

```
SQL> SELECT INITCAP(FIRST_NAME) AS "First Name",
  2         INITCAP(LAST_NAME) AS "Last Name",
  3         LENGTH(FIRST_NAME || LAST_NAME) AS "Name Length"
  4  FROM employees
  5  WHERE UPPER(SUBSTR(FIRST_NAME, 1, 1)) IN ('J', 'A', 'M')
  6  ORDER BY LAST_NAME;

First Name           Last Name                 Name Length
-------------------- ------------------------- -----------
Mozhe                Atkinson                           13
Amit                 Banda                               9
Alexis               Bull                               10
Anthony              Cabrio                             13
John                 Chen                                8
Julia                Dellinger                          14
Jennifer             Dilly                              13
Alberto              Errazuriz                          16
Jean                 Fleaur                             10
Adam                 Fripp                               9
Michael              Hartstein                          16
```

Q-5

Write a query that produces the following for each employee:
<employee last name> earns <salary> monthly

A

```
SQL> SELECT LAST_NAME || ' earns' || SALARY || ' monthly' AS "Salaey Info" from employees;

Salaey Info
--------------------------------------------------------------------------------
King earns24000 monthly
Kochhar earns17000 monthly
De Haan earns17000 monthly
Hunold earns9000 monthly
Ernst earns6000 monthly
Austin earns4800 monthly
Pataballa earns4800 monthly
Lorentz earns4200 monthly
Greenberg earns12008 monthly
Faviet earns9000 monthly
Chen earns8200 monthly
```

Q-6

Display the name, date, number of months employed and day of the week on which the employee has started. Order the results by the day of the week starting with Monday.

A

```
SQL> select first_name || ' ' || last_name as "Employee Name",
  2         hire_date,
  3         months_between(sysdate, hire_date) as "Months Employed",
  4         to_char(hire_date, 'Day') as "Day of the Week"
  5  from employees
  6  order by to_char(hire_date, 'D');

Employee Name                                  HIRE_DATE Months Employed Day of the Week
---------------------------------------------- --------- --------------- ---------------
Valli Pataballa                                05-FEB-06      228.239107 Sunday
James Landry                                   14-JAN-07      216.948784 Sunday
Douglas Grant                                  13-JAN-08      204.981042 Sunday
Peter Tucker                                   30-JAN-05      240.432655 Sunday
Allan McEwen                                   01-AUG-04      246.368139 Sunday
Adam Fripp                                     10-APR-05      238.077816 Sunday
Matthew Weiss                                  18-JUL-04      246.819752 Sunday
Sigal Tobias                                   24-JUL-05      234.626203 Sunday
Peter Vargas                                   09-JUL-06      223.110074 Sunday
Jack Livingston                                23-APR-06      225.658461 Sunday
Alexis Bull                                    20-FEB-05      239.755236 Sunday
```

Q-7 Display the date of emp in a format that appears as Seventh of June 1994 12:00:00 AM.

A

```
SQL> SELECT TO_CHAR(hiredate, 'DDspth "of" Month YYYY HH:MI:SS AM') AS "Formatted Date"
  2  FROM employee;

Formatted Date
-----------------------------------------------------------------------
NINTH of August    2096 12:00:00 AM
FOURTEENTH of March     2096 12:00:00 AM
THIRTIETH of November  2095 12:00:00 AM
SECOND of October   2097 12:00:00 AM
FIRST of January   2098 12:00:00 AM
TWENTY-SIXTH of September 2097 12:00:00 AM
FIFTEENTH of July      2097 12:00:00 AM

7 rows selected.
```

Q-8 Write a query to calculate the annual compensation of all employees (sal +comm.).

A

```
SQL> select employee_id,
  2         salary,
  3         commission_pct,
  4         (salary * 12 + (nvl(commission_pct, 0) * salary * 12)) as "Annual Compensation"
  5  from employees;

EMPLOYEE_ID     SALARY COMMISSION_PCT Annual Compensation
----------- ---------- -------------- -------------------
        100      24000                             288000
        101      17000                             204000
        102      17000                             204000
        103       9000                             108000
        104       6000                              72000
        105       4800                              57600
        106       4800                              57600
        107       4200                              50400
        108      12008                             144096
        109       9000                             108000
        110       8200                              98400
```

	Practical – 5
5.	Displaying data from Multiple Tables (join)
Q-1	Find advisors for students in departments whose names start with "Comp".
A	(see below)

```
SQL> SELECT s.NAME AS Student_Name, a.I_ID AS Advisor_ID, s.DEPT_NAME
  2  FROM student s
  3  JOIN advisor a ON s.ID = a.S_ID
  4  WHERE s.DEPT_NAME LIKE 'Comp%';

STUDENT_NAME         ADVIS DEPT_NAME
-------------------- ----- --------------------
McGarr               90376 Comp. Sci.
Konno                36897 Comp. Sci.
Hird                 58558 Comp. Sci.
Dubink               78699 Comp. Sci.
Batano               16807 Comp. Sci.
Maglioni             65931 Comp. Sci.
Haigh                6569  Comp. Sci.
Jawad                19368 Comp. Sci.
Havill               50885 Comp. Sci.
Guyer                48507 Comp. Sci.
Wakamiya             74420 Comp. Sci.
```

Q-2	List courses that are prerequisites for courses with titles containing 'Geology'.
A	(see below)

```
SQL> select distinct p.course_id, c.title as course_title, p.prereq_id
  2  from prereq p
  3  join course c on p.course_id = c.course_id
  4  where c.title like '%Accounting%';

COURSE_I COURSE_TITLE                                       PREREQ_I
-------- -------------------------------------------------- --------
376      Cost Accounting                                    130
486      Accounting                                         371
```

Q-3	Find instructors teaching courses with "Programming" in the course title or classroom number.
A	

```
SQL> select distinct i.id, i.name, c.title, s.building, s.room_number
  2  from instructor i
  3  join teaches t on i.id = t.id
  4  join section s on t.course_id = s.course_id and t.sec_id = s.sec_id
  5  join course c on t.course_id = c.course_id
  6  where c.title like '%programming%' ;

no rows selected
```

Q-4

Write a query to display the last name, department number, and department name for all employees.

A

```
SQL> select e.last_name, e.department_id, d.department_name
  2  from employees e
  3  join departments d on e.department_id = d.department_id;

LAST_NAME                 DEPARTMENT_ID DEPARTMENT_NAME
------------------------- ------------- ------------------------------
Whalen                               10 Administration
Hartstein                            20 Marketing
Fay                                  20 Marketing
Raphaely                             30 Purchasing
Khoo                                 30 Purchasing
Baida                                30 Purchasing
Tobias                               30 Purchasing
Himuro                               30 Purchasing
Colmenares                           30 Purchasing
Mavris                               40 Human Resources
Weiss                                50 Shipping
```

Q-5

Display the employee last name and department name for all employees who have an a (lowercase) in their last names.

A

	<pre>SQL> select e.last_name, d.department_name 2 from employees e 3 join departments d on e.department_id = d.department_id 4 where lower(e.last_name) like '%a%'; LAST_NAME DEPARTMENT_NAME ------------------------- ------------------------------ Whalen Administration Hartstein Marketing Fay Marketing Baida Purchasing Tobias Purchasing</pre>
Q-6	Display the employee last name and employee number along with their manager's last name and manager number. Label the columns Employee, Emp#, Manager, and Mgr#, respectively.
A	<pre>SQL> select e.last_name as employee, e.employee_id as emp#, 2 m.last_name as manager, e.manager_id as mgr# 3 from employees e 4 join employees m on e.manager_id = m.employee_id; EMPLOYEE EMP# MANAGER MGR# ------------------------- ---------- ------------------------- ---------- Kochhar 101 King 100 De Haan 102 King 100 Raphaely 114 King 100 Weiss 120 King 100 Fripp 121 King 100 Kaufling 122 King 100 Vollman 123 King 100 Mourgos 124 King 100</pre>
Q-7	Modify (6) to display all employees including King, who has no manager. Order the results by the employee number
A	

	`` SQL> select e.last_name as employee, e.employee_id as emp#, 2 m.last_name as manager, e.manager_id as mgr# 3 from employees e 4 left join employees m on e.manager_id = m.employee_id 5 order by e.employee_id; EMPLOYEE EMP# MANAGER MGR# ------------------------- ---------- ------------------------- ---------- King 100 Kochhar 101 King 100 De Haan 102 King 100 Hunold 103 De Haan 102 Ernst 104 Hunold 103 Austin 105 Hunold 103 Pataballa 106 Hunold 103 Lorentz 107 Hunold 103 Greenberg 108 Kochhar 101 Faviet 109 Greenberg 108 Chen 110 Greenberg 108 ```
Q-8	Create a query to display the name and hire date of any employee hired after employee Jha.
A	``` SQL> select e.last_name, e.hire_date 2 from employees e 3 where e.hire_date > (select hire_date from employees where last_name = 'Davies'); LAST_NAME HIRE_DATE ------------------------- --------- Kochhar 21-SEP-05 Hunold 03-JAN-06 Ernst 21-MAY-07 Austin 25-JUN-05 Pataballa 05-FEB-06 Lorentz 07-FEB-07 Chen 28-SEP-05 Sciarra 30-SEP-05 Urman 07-MAR-06 Popp 07-DEC-07 Baida 24-DEC-05 ```
Q-9	Display the names and hire dates for all employees who were hired before their managers, along with their manager's names and hire dates. Label the columns Employee, Emp Hired, Manager, and Mgr Hired, respectively.
A	

<table>
<tr><td></td><td>

```
SQL> select e.last_name as Employee, e.hire_date as emp_hire ,m.last_name as manager ,m.hire_date as man_hire
  2  from employees e
  3  join employees m  on  e.manager_id = m.employee_id
  4  where e.hire_date >m.hire_date;

EMPLOYEE                  EMP_HIRE  MANAGER                   MAN_HIRE
------------------------- --------- ------------------------- ---------
Kochhar                   21-SEP-05 King                      17-JUN-03
Weiss                     18-JUL-04 King                      17-JUN-03
Fripp                     10-APR-05 King                      17-JUN-03
Vollman                   10-OCT-05 King                      17-JUN-03
Mourgos                   16-NOV-07 King                      17-JUN-03
Russell                   01-OCT-04 King                      17-JUN-03
Partners                  05-JAN-05 King                      17-JUN-03
Errazuriz                 10-MAR-05 King                      17-JUN-03
Cambrault                 15-OCT-07 King                      17-JUN-03
Zlotkey                   29-JAN-08 King                      17-JUN-03
Hartstein                 17-FEB-04 King                      17-JUN-03
```

</td></tr>
<tr><td>Q-10</td><td>Create a unique listing of all jobs that are in department 80. Include the location of the department in the output.</td></tr>
<tr><td>A</td><td>

```
SQL> select distinct j.job_title, d.location_id
  2  from employees e
  3  join jobs j on e.job_id = j.job_id
  4  join departments d on e.department_id = d.department_id
  5  where e.department_id = 80;

JOB_TITLE                           LOCATION_ID
----------------------------------- -----------
Sales Manager                              2500
Sales Representative                       2500
```

</td></tr>
</table>

Annexure V: Perform following queries on JOIN for further practice work

<table>
<tr><td>Q-1</td><td>list employees, their country, and region</td></tr>
<tr><td>A</td><td>

```
SQL> SELECT e.emp_no, e.emp_name, l.city AS location, c.country_id, r.region_name
  2  FROM employee e
  3  JOIN departments d ON e.dept_no = d.department_id
  4  JOIN locations l ON d.location_id = l.location_id
  5  JOIN countries c ON l.country_id = c.country_id
  6  JOIN regions r ON c.region_id = r.region_id;

    EMP_NO EMP_NAME                       LOCATION                       CO REGION_NAME
---------- ------------------------------ ------------------------------ -- -------------------------
       106 Sneha                          Seattle                        US Americas
       105 Anita                          Seattle                        US Americas
       103 Adama                          Toronto                        CA Americas
```

</td></tr>
</table>

Q-2	retrieve employees who have changed jobs
A	

```
SQL> SELECT DISTINCT e1.emp_name, e1.job_id
  2  FROM employee e1
  3  JOIN employee e2 ON e1.job_id = e2.job_id AND e1.emp_no <> e2.emp_no;

EMP_NAME                       JOB_ID
------------------------------ ---------------
Anita                          comp_op
Aman                           comp_op
```

Q-3	retrieve all departments and employees, including departments without employees

```
SQL> SELECT d.dept_name, e.emp_no, e.emp_name
  2  FROM department d
  3  LEFT JOIN employee e ON d.dept_name = e.dept_name;

DEPT_NAME                EMP_NO EMP_NAME
-------------------- ---------- ------------------------------
Athletics
Math
Astronomy
Elec._Eng.
Pol._Sci.
Civil_Eng.
Marketing
Biology
Comp._Sci.
Cybernetics
English
```

Q-4	retrieve all employees and their managers (self join)

A

```
SQL> SELECT e.emp_name AS Employee, e.emp_no AS Emp_ID,
  2         m.emp_name AS Manager, m.emp_no AS Mgr_ID
  3  FROM employee e
  4  LEFT JOIN employee m ON e.manager_id = m.emp_no;

EMPLOYEE                           EMP_ID MANAGER                            MGR_ID
------------------------------ ---------- ------------------------------ ----------
Smith                                 101 Anita                                 105
Adama                                 103 Anita                                 105
Sneha                                 106 Anita                                 105
Anita                                 105 Anamika                               107
Snehal                                102
Anamika                               107
Aman                                  104

7 rows selected.
```

Q-5 retrieve all employees who have the same job title as their manager

A

```
SQL> SELECT e.emp_no, e.emp_name, e.job_id, e.manager_id, m.emp_no AS Manager_ID, m.emp_name AS Manager, m.job_id AS Manager_Job_ID
  2  FROM employee e
  3  JOIN employee m ON e.manager_id = m.emp_no;

    EMP_NO EMP_NAME                       JOB_ID          MANAGER_ID MANAGER_ID MANAGER                        MANAGER_JOB_ID
---------- ------------------------------ --------------- ---------- ---------- ------------------------------ ---------------
       103 Adama                          mk_mgr                 105        105 Anita                          comp_op
       106 Sneha                          fi_acc                 105        105 Anita                          comp_op
       101 Smith                          fi_mgr                 105        105 Anita                          comp_op
       105 Anita                          comp_op                107        107 Anamika                        it_prog
```

Q-6 retrieve departments without employees

A

```
SQL> SELECT d.dept_name
  2  FROM department d
  3  LEFT JOIN employee e ON d.dept_name = e.dept_name
  4  WHERE e.emp_no IS NULL;

DEPT_NAME
--------------------
Athletics
Math
Astronomy
Elec._Eng.
Pol._Sci.
Civil_Eng.
Marketing
Biology
Comp._Sci.
Cybernetics
English
```

Q-7 retrieve job changes along with new and old departments

A	``` SQL> SELECT jh.employee_id, e.emp_name, 2 jh.start_date AS Old_Job_Start, e.hiredate AS New_Job_Start, 3 jh.department_id AS Old_Department, e.dept_no AS New_Department 4 FROM job_history jh 5 JOIN employee e ON jh.employee_id = e.emp_no; EMPLOYEE_ID EMP_NAME OLD_JOB_S NEW_JOB_S OLD_DEPARTMENT NEW_DEPARTMENT ----------- ------------------------------ --------- --------- -------------- -------------- 101 Smith 21-SEP-97 09-AUG-96 110 1 101 Smith 28-OCT-01 09-AUG-96 110 1 102 Snehal 13-JAN-01 14-MAR-96 60 25 ```
Q-8	retrieve employees working in locations in the usa
A	``` SQL> SELECT e.emp_no, e.emp_name, l.city 2 FROM employee e 3 JOIN departments d ON e.dept_no = d.department_id 4 JOIN locations l ON d.location_id = l.location_id 5 WHERE l.country_id = 'US'; EMP_NO EMP_NAME CITY ---------- ------------------------------ ------------------------------ 106 Sneha Seattle 105 Anita Seattle ```

<table>
<tr><td></td><td>Practical – 6</td></tr>
<tr><td>6.</td><td>To apply the concept of Aggregating Data using Group functions.</td></tr>
<tr><td>Q-1</td><td>Display the highest, lowest, sum, and average salary of all employees. Label the columns Maximum, Minimum, Sum, and Average, respectively. Round your results to the nearest whole number.</td></tr>
<tr><td>A</td><td>

```
SQL>  SELECT ROUND(MAX(salary)) AS "Maximum",
  2          ROUND(MIN(salary)) AS "Minimum",
  3          ROUND(SUM(salary)) AS "Sum",
  4          ROUND(AVG(salary)) AS "Average"
  5  FROM employees;

   Maximum    Minimum        Sum    Average
---------- ---------- ---------- ----------
     24000       2100     691416       6462
```

</td></tr>
<tr><td>Q-2</td><td>Write a query that displays the difference between the highest and lowest salaries. Label the column DIFFERENCE.</td></tr>
<tr><td>A</td><td>

```
SQL> SELECT MAX(salary) - MIN(salary) AS "DIFFERENCE"
  2  FROM employees;

DIFFERENCE
----------
     21900
```

</td></tr>
<tr><td>Q-3</td><td>Create a query that will display the total number of employees and, of that total, the number of employees hired in 1995, 1996, 1997, and 1998</td></tr>
</table>

A

```
SQL> select count(employee_id) as "Total employees hired in 2002,2004,2006 and 1995 " from employees where to_char(hire_date,'yy') in('02','04',
'06','95');

Total employees hired in 2002,2004,2006 and 1995
-------------------------------------------------
                                               41
```

Q-4 Find the average salaries for each department without displaying the respective department numbers.

A

```
SQL> SELECT ROUND(AVG(salary)) AS "Average Salary"
  2  FROM employees
  3  GROUP BY department_id;

Average Salary
--------------
         19333
          5760
          8601
          4150
          3476
          8956
          7000
          4400
          9500
          6500
         10000

Average Salary
--------------
         10154

12 rows selected.
```

Q-5 Write a query to display the total salary being paid to each job title, within each department.

A

```
SQL> SELECT job_id, department_id, SUM(salary) AS "Total Salary"
  2  FROM employees
  3  GROUP BY job_id, department_id
  4  ORDER BY department_id, job_id;

JOB_ID     DEPARTMENT_ID Total Salary
---------- ------------- ------------
AD_ASST               10         4400
MK_MAN                20        13000
MK_REP                20         6000
PU_CLERK              30        13900
PU_MAN                30        11000
HR_REP                40         6500
SH_CLERK              50        64300
ST_CLERK              50        55700
ST_MAN                50        36400
IT_PROG               60        28800
PR_REP                70        10000
```

```
SQL> select job_id, dept_name, SUM(emp_sal) AS total_salary
  2  from employee
  3  group by job_id, dept_name;

JOB_ID          DEPT_NAME                      TOTAL_SALARY
--------------- ------------------------------ ------------
fi_mgr          Machine Learning                        800
lec             Data Science                           1600
mk_mgr          Machine Learning                       1100
comp_op         Virtual Reality                        3000
comp_op         Big Data Analytics                     5000
fi_acc          Big Data Analytics                     2450
it_prog         Artificial Intelligence                2975

7 rows selected.
```

Q-6	Find the average salaries > 2000 for each department without displaying the respective department numbers.

A	(see code below)
Q-7	Display the job and total salary for each job with a total salary amount exceeding 3000 and sorts the list by the total salary.
A	(see code below)

```
SQL> SELECT ROUND(AVG(salary)) AS "Average Salary"
  2  FROM employees
  3  GROUP BY department_id
  4  HAVING AVG(salary) > 2000;

Average Salary
--------------
         19333
          5760
          8601
          4150
          3476
          8956
          7000
          4400
          9500
          6500
         10000

Average Salary
--------------
         10154

12 rows selected.
```

```
SQL> SELECT job_id, SUM(salary) AS "Total Salary"
  2  FROM employees
  3  GROUP BY job_id
  4  HAVING SUM(salary) > 3000
  5  ORDER BY SUM(salary) DESC;

JOB_ID     Total Salary
---------- ------------
SA_REP           250500
SH_CLERK          64300
SA_MAN            61000
ST_CLERK          55700
FI_ACCOUNT        39600
ST_MAN            36400
AD_VP             34000
IT_PROG           28800
AD_PRES           24000
PU_CLERK          13900
MK_MAN            13000

JOB_ID     Total Salary
---------- ------------
AC_MGR            12008
FI_MGR            12008
PU_MAN            11000
PR_REP            10000
AC_ACCOUNT         8300
HR_REP             6500
MK_REP             6000
AD_ASST            4400

19 rows selected.
```

Practical – 7

7. **To solve queries using the concept of sub query.**

Q-1 Find the total credits for each student and include their department.

A

```
SQL> select s.id, s.name, s.dept_name,
  2  (select sum(c.credits)
  3  from takes t
  4  join course c on t.course_id = c.course_id
  5  where t.id = s.id) as total_credits
  6  from student s;

ID    NAME                 DEPT_NAME            TOTAL_CREDITS
----- -------------------- -------------------- -------------
61065 Jovicic              Civil Eng.
107   Shabuno              Math
11453 Yamashita            Astronomy
53805 Ludwig               Cybernetics
39241 Solar                Mech. Eng.
32886 Damas                Psychology
40080 Llam                 Civil Eng.
22142 Gerstend             History
94257 Unger                Languages
75513 Griffin              Statistics
99268 Makarychev           Elec. Eng.
```

Q-2 Find instructors who earn more than the average salary.

A

```
SQL> select name, salary
  2  from instructor
  3  where salary > (select avg (salary) from instructor);

NAME                     SALARY
-------------------- ----------
McKinnon               94333.99
Mird                  119921.41
Luo                    88791.45
Levine                 89805.83
Shuming               118812.99
Sullivan               90038.09
Voronina              133256.19
Arias                 109791.55
Mingoz                105311.38
Yazdi                  98333.65
Kenje                 106554.73
```

Q-3 List courses offered by the 'Physics' department.

A

	<pre>SQL> select course_id, title 2 from course 3 where dept_name = 'Physics'; COURSE_I TITLE -------- -- 612 Mobile Computing 376 Cost Accounting 959 Bacteriology 267 Hydraulics 436 Stream Processing 731 The Music of Donovan 130 Differential Geometry 239 The Music of the Ramones 580 The Music of Dave Edmunds 443 Journalism 10 rows selected.</pre>
Q-4	Find students who have taken a course titled "Database Systems".
A	<pre>SQL> select distinct s.id, s.name 2 from student s 3 where s.id in (select t.id 4 from takes t 5 join course c on t.course_id = c.course_id 6 where c.title = 'Database System Concepts'); no rows selected</pre>
Q-5	List the names of instructors in the 'Physics' department.
A	<pre>SQL> select name from instructor where dept_name = 'Physics'; NAME -------------------- Shuming Voronina</pre>
Q-6	Find students enrolled in the course "893".

A

```
SQL> SELECT s.ID, s.NAME
  2  FROM student s
  3  WHERE s.ID IN (SELECT ID FROM takes WHERE COURSE_ID = '893');

ID    NAME
----- --------------------
10269 Hilberg
108   Dhav
10834 More
1087  Roses
11152 Al-Tahat
11194 El-Helw
11966 Kowe
12078 Knutson
1220  Hito
12214 Morales
12236 Bricker
```

Q-7 Find instructors who earn more than all instructors in the 'Mathematics' department.

A

```
SQL> select name, salary
  2  from instructor
  3  where salary > all (select salary from instructor where dept_name = 'Mathematics');

NAME                     SALARY
-------------------- ----------
Lembr                  32241.56
Konstantinides          32570.5
Vicentino              34272.67
Kean                   35023.18
Morris                 43770.36
Hau                    43966.29
Queiroz                45538.32
Yin                    46397.59
Ullman                  47307.1
Gutierrez              47576.06
Desyl                  48803.38
```

Q-8 Find instructors who earn more than at least one instructor in the 'Physics' department.

A

```
SQL> select name, salary
  2  from instructor
  3  where salary > (
  4  select MIN(salary)
  5  from instructor
  6  where dept_name = 'Physics');

NAME                     SALARY
-------------------- ----------
Mird                  119921.41
Voronina              133256.19
Wieland               124651.41
```

Q-9 Find courses where the total number of enrolled students exceeds 10.

A

```
SQL> select course_id
  2  from takes
  3  where course_id in (
  4  select course_id
  5  from takes
  6  group by course_id
  7  having count(distinct id) > 10);
```

Q-10 Find departments with more than 5 employees.

A

```
SQL> select dept_name
  2  from department
  3  where dept_name in (
  4  select dept_name
  5  from employee
  6  group by dept_name
  7  having count(*) > 5);

no rows selected
```

Q-11 Increase the salary of employees who earn less than the average salary.

A	`SQL> update employee` `  2  set emp_sal = emp_sal * 1.10` `  3  where emp_sal < (select avg(emp_sal) from employee);` `3 rows updated.`
Q-12	Remove employees who have no manager assigned.
A	`SQL> delete from employee` `  2  where manager_id is null;` `0 rows deleted.`
Q-13	Find employees working in the same department as the employee 'Steven King'.
A	`SQL> select emp_name` `  2  from employee` `  3  where dept_no = (select dept_no from employee where emp_name = 'Steven King');` `no rows selected`

	Practical – 8
8.	**To Implement Single-row functions.**
Q-1	Develop a query to grant all privileges of employees table to new user.
A	`SQL> grant all privileges on c##yash.employees to c##new_user;` `Grant succeeded.`
Q-2	Develop a query to grant some privileges of employees table to new user.
A	`SQL> grant select, insert, update on c##yash.employees to c##new_user;` `Grant succeeded.`
Q-3	Develop a query to revoke all privileges of employees table from user.
A	`SQL> revoke all privileges on c##yash.employees from c##new_user;` `Revoke succeeded.`
Q-4	Develop a query to revoke some privileges of employees table from user.
A	`SQL> revoke insert, update on c##yash.employees from c##new_user;` `Revoke succeeded.`

	Practical – 9
9.	**Add and Remove constraint**
Q-1	Add primary key constraint on job_id in job table.
A	SQL> ALTER TABLE jobs_copy 2 ADD CONSTRAINT pk_job PRIMARY KEY (job_id); Table altered.
Q-2	Add foreign key constraint on employee table referencing job table.
A	SQL> ALTER TABLE employees_copy 2 ADD CONSTRAINT fk_employee_job_copy FOREIGN KEY (job_id) 3 REFERENCES jobs(job_id); Table altered.
Q-3	Add composite primary key on lock table (lock table does not exist, while creating table add composite key)
A	SQL> CREATE TABLE lock_table (2 lock_id NUMBER(5), 3 user_id NUMBER(5), 4 lock_name VARCHAR2(50), 5 PRIMARY KEY (lock_id, user_id) 6); Table created.
Q-4	Remove primary key constraint on job_id
A	SQL> ALTER TABLE employee_copy DROP CONSTRAINT FK_EMP_JOB_COPY; Table altered.

Q-5	Remove foreign key constraint on employee table
A	`SQL> ALTER TABLE jobs_copy DROP CONSTRAINT PK_JOB_COPY;` `Table altered.`

	Practical – 10
10.	**To perform basic PL/SQL blocks**
Q-1	Write a PL-SQL block for checking weather a given year is a Leap year or not

A

```
declare
    year number := &year;
begin
    if(mod(year, 4) = 0 AND mod(year, 100) != 0) OR (mod(year, 400) = 0) then
        dbms_output.put_line(year || ' is a leap year');
    else
        dbms_output.put_line(year || ' is not a leap year');
    end if;
end;
/
```

Output :

```
SQL> @23ce142_pra_10_1
Enter value for year: 2024
old   5:      year := &year;
new   5:      year := 2024;
2024 is a leap year

PL/SQL procedure successfully completed.

SQL>
```

Q-2	Find out whether given string is palindrome or not using for a while and simple loop
A	**Using For loop :**

```
declare
   str VARCHAR2(50) := '&string';
   rev_str VARCHAR2(50) := '';
   len NUMBER;
begin
   len := LENGTH(str);
   for i in reverse 1..len loop
      rev_str := rev_str || SUBSTR(str, i, 1);
   end loop;

   if str = rev_str THEN
      dbms_output.put_line(str || ' is a Palindrome');
   else
      dbms_output.put_line(str || ' is NOT a Palindrome');
   end if;
end;
/
```

Output :

```
SQL> @plsql
Enter value for str1: Yash
old   8:      str1 := '&str1';
new   8:      str1 := 'Yash';
Yash is NOT a Palindrome.

PL/SQL procedure successfully completed.
```

Using While loop :

```
declare

        str varchar2(50) := '&string';
        rev_str varchar(50) := '';
        len number;
        i number;

begin

        len := length(str);
        i := len;

        while i > 0 loop
                rev_str := rev_str || SUBSTR(str, i, 1);
                i := i - 1;
        end loop;

        if str = rev_str then
                dbms_output.put_line(str || 'is a palindrome ');

        else
                dbms_output.put_line(str || 'is not a palindrome');
        end if;
end;
/
```

Output:

```
SQL> @10_while
Enter value for str1: nayan
old   8:          str1:='&str1';
new   8:          str1:='nayan';
nayanis a Palindrome.
```

Using Simple Loop :

```
declare
   str VARCHAR2(50) := '&string';
   rev_str VARCHAR2(50) := '';
   len NUMBER;
   i NUMBER;
begin
   len := LENGTH(str);
   i := len;

   loop
      exit when i = 0;
      rev_str := rev_str || SUBSTR(str, i, 1);
      i := i - 1;
   end loop;

   if str = rev_str then
      dbms_output.put_line(str || ' is a Palindrome');
   else
      dbms_output.put_line(str || ' is NOT a Palindrome');
   end if;
end;
/
```

Output:

```
SQL> @pra10_simple
Enter value for str1: ABBA
old   8:         str1:='&str1';
new   8:         str1:='ABBA';
ABBAis a Palindrome.
```

	Practical – 11
11.	**To understand the concept of “select into” and “% type” attribute.**
Q-1	Create an EMPLOYEES table that is a replica of the EMP table. Add a new column, STARS, of VARCHAR2 data type and length of 50 to the EMPLOYEES table for storing asterisk (*). Create a PL/SQL block that rewards an employee by appending an asterisk in the STARS column for every Rs1000/- of the employee’s salary. For example, if the employee has a salary amount of Rs8000/-, the string of asterisks should contain eight asterisks. If the employee has a salary amount of Rs12500/-, the string of asterisks should contain 13 asterisks. Update the STARS column for the employee with the string of asterisks.
A	See code and output below

```
declare
      v_empno emp.employee_id%type := to_number('&v_empno');
      v_sal emp.salary%type;
       v_asterisk VARCHAR2(50) := '';
begin
      select nvl(round(salary / 1000), 0) into v_sal from emp where employee_id =  v_empno;

      for i in 1 .. v_sal loop
            v_asterisk := v_asterisk || '*';
      end loop;

update emp set stars = v_asterisk where employee_id = v_empno;

commit;

dbms_output.put_line('Updated employee ' || v_empno || ' with stars: ' || v_asterisk);
end;
/
```

Output :

```
SQL> @pra_11_1
Enter value for v_empno: 101
old   2:     v_empno emp.employee_id%type := to_number('&v_empno');
new   2:     v_empno emp.employee_id%type := to_number('101');
Updated employee 101 with stars: *****************

PL/SQL procedure successfully completed.
```

Annexure XI: Performing using cursor

Q-1

Create an EMPLOYEES table that is a replica of the EMP table. Add a new column, STARS, of VARCHAR2 data type and length of 50 to the EMPLOYEES table for storing asterisk (*).

Create a PL/SQL block that rewards an employee by appending an asterisk in the STARS column for every Rs1000/- of the employee's salary. For example, if the employee has a salary amount of Rs8000/-, the string of asterisks should contain eight asterisks. If the employee has a salary amount of Rs12500/-, the string of asterisks should contain 13 asterisks.

Update the STARS column for the employee with the string of asterisks.

Using Cursor

A

```
declare
   cursor emp_cursor IS
      select salary
      from emp
      where employee_id = TO_NUMBER('&v_empno');

   v_empno emp.employee_id%TYPE := TO_NUMBER('&v_empno');
   v_sal emp.salary%TYPE;
   v_asterisk VARCHAR2(50) := '';

begin

   open emp_cursor;
   fetch emp_cursor INTO v_sal;

   if emp_cursor%found then
      v_sal := nvl(round(v_sal / 1000), 0);

      for i in 1 .. v_sal loop
         v_asterisk := v_asterisk || '*';
      end loop;

      update emp set stars = v_asterisk where employee_id = v_empno;
      commit;

      dbms_output.put_line('Updated employee ' || v_empno || ' with stars: ' || v_asterisk);
   else
      dbms_output.put_line('Employee ID ' || v_empno || ' not found.');
   end if;

   close emp_cursor;
end;
/
```

c

Output:-

```
SQL> @pra_11_2
Enter value for v_empno: 101
old   5:          where employee_id = TO_NUMBER('&v_empno');
new   5:          where employee_id = TO_NUMBER('101');
Enter value for v_empno: 5000
old   7:      v_empno emp.employee_id%TYPE := TO_NUMBER('&v_empno');
new   7:      v_empno emp.employee_id%TYPE := TO_NUMBER('5000');
Updated employee 5000 with stars: *****************

PL/SQL procedure successfully completed.
```

Practical – 12

12. **To perform the concept of cursor**

A) Display all the information of EMP table using %ROWTYPE.

A

Code:

```
DECLARE

        v_emp employees%ROWTYPE;

        CURSOR emp_cursor IS
                SELECT * FROM employees;

        BEGIN
                OPEN emp_cursor;
                LOOP
                        FETCH emp_cursor INTO v_emp;
                        EXIT WHEN emp_cursor%NOTFOUND;
                        DBMS_OUTPUT.PUT_LINE(TO_CHAR(v_emp.employee_id) || ' -- ' ||v_emp.FIRST_NAME|| '  '
||v_emp.LAST_NAME|| '  ' ||v_emp.EMAIL|| '  ' ||v_emp.PHONE_NUMBER|| '  ' ||v_emp.HIRE_DATE|| '  '
||v_emp.JOB_ID|| '  ' ||v_emp.SALARY|| '  ' ||v_emp.COMMISSION_PCT|| '  ' ||v_emp.MANAGER_ID|| '  '
||v_emp.DEPARTMENT_ID);

                END LOOP;
                CLOSE emp_cursor;
END;
```

Output:

```
SQL> set serveroutput on;
SQL> @ F:\CSPIT-CE\SEM-4\DBMS\pct_12_cursor_rowtype_example.sql
 18  /
198 -- Donald  OConnell  DOCONNEL  650.507.9833  21-JUN-07  SH_CLERK  2600
124  50
199 -- Douglas  Grant  DGRANT  650.507.9844  13-JAN-08  SH_CLERK  2600    124
50
200 -- Jennifer  Whalen  JWHALEN  515.123.4444  17-SEP-03  AD_ASST  4400    101
10
201 -- Michael  Hartstein  MHARTSTE  515.123.5555  17-FEB-04  MK_MAN  13000
100  20
202 -- Pat  Fay  PFAY  603.123.6666  17-AUG-05  MK_REP  6000    201  20
203 -- Susan  Mavris  SMAVRIS  515.123.7777  07-JUN-02  HR_REP  6500    101  40
204 -- Hermann  Baer  HBAER  515.123.8888  07-JUN-02  PR_REP  10000    101  70
205 -- Shelley  Higgins  SHIGGINS  515.123.8080  07-JUN-02  AC_MGR  12008    101
110
206 -- William  Gietz  WGIETZ  515.123.8181  07-JUN-02  AC_ACCOUNT  8300    205
110
100 -- Steven  King  SKING  515.123.4567  17-JUN-03  AD_PRES  24000      90
101 -- Neena  Kochhar  NKOCHHAR  515.123.4568  21-SEP-05  AD_VP  17000    100
90
102 -- Lex  De Haan  LDEHAAN  515.123.4569  13-JAN-01  AD_VP  17000    100  90
103 -- Alexander  Hunold  AHUNOLD  590.423.4567  03-JAN-06  IT_PROG  9000    102
60
104 -- Bruce  Ernst  BERNST  590.423.4568  21-MAY-07  IT_PROG  6000    103  60
105 -- David  Austin  DAUSTIN  590.423.4569  25-JUN-05  IT_PROG  4800    103  60
106 -- Valli  Pataballa  VPATABAL  590.423.4560  05-FEB-06  IT_PROG  4800    103
60
107 -- Diana  Lorentz  DLORENTZ  590.423.5567  07-FEB-07  IT_PROG  4200    103
60
108 -- Nancy  Greenberg  NGREENBE  515.124.4569  17-AUG-02  FI_MGR  12008    101
```

B) Create a PL/SQL block that does the following: In a PL/SQL block, retrieve the name, salary, and MANAGER ID of the employees working in the particular department. Take Department Id from user.

If the salary of the employee is less than 1000 and if the manager ID is either 7902 or 7839, display the message <> Due for a raise. Otherwise, display the message <> Not due for a raise.

A

Code:

```
declare

v_dept_no number :=(&v_dept_no);

begin

for i in (select last_name,salary from employees where department_id=v_dept_no and manager_id=122 or
manager_id=149)
loop

        if(i.salary<1000) then
                dbms_output.put_line(i.last_name||' due for raise ');
        else
                dbms_output.put_line(i.last_name||' not due for raise ');

        end if;
end loop;
end;
/
```

Output:

```
SQL> @ F:\CSPIT-CE\SEM-4\DBMS\pct_12_2.sql
Enter value for v_dept_no: 50
old   3: v_dept_no number :=(&v_dept_no);
new   3: v_dept_no number :=(50);
Mallin not due for raise
Rogers not due for raise
Gee not due for raise
Philtanker not due for raise
Abel not due for raise
Hutton not due for raise
Taylor not due for raise
Livingston not due for raise
Grant not due for raise
Johnson not due for raise
Chung not due for raise
Dilly not due for raise
Gates not due for raise
Perkins not due for raise

PL/SQL procedure successfully completed.
```

C)	In a loop, use a cursor to retrieve the department number and the department name from the DEPT table for those departments whose DEPT_ID is less than 100. Pass the department number to another cursor to retrieve from the EMP table the details of employee name, job, hire date, and salary of those employees whose EMP_NO is less than 7566 and who work in that department

A

Code:

```
DECLARE
        CURSOR dept_cursor IS
        SELECT department_id,department_name   FROM     departments
        WHERE department_id < 100     ORDER BY    department_id;
        CURSOR emp_cursor(v_deptno      NUMBER) IS
        SELECT last_name,manager_id,hire_date,salary
        FROM    emp
        WHERE   department_id = v_deptno
        AND employee_id < 7566;
        v_current_deptno departments.department_id%TYPE;
        v_current_dname  departments.department_name%TYPE;
        v_ename emp.last_name%TYPE;
        v_job emp.manager_id%TYPE;
        v_hiredate emp.hire_date%TYPE;
        v_sal emp.salary%TYPE;
        v_line  varchar2(100);
BEGIN
        v_line := '                                                                                                    ';
        OPEN dept_cursor;
         LOOP
                FETCH dept_cursor INTO v_current_deptno,v_current_dname;
                EXIT WHEN dept_cursor%NOTFOUND;
                DBMS_OUTPUT.PUT_LINE ('Department Number : ' ||  v_current_deptno ||  '  Department Name : ' || v_current_dname);
                DBMS_OUTPUT.PUT_LINE(v_line);
                IF emp_cursor%ISOPEN THEN
                       CLOSE emp_cursor;
               END IF;
                       OPEN emp_cursor (v_current_deptno);
                LOOP
                       FETCH emp_cursor INTO  v_ename,v_job,v_hiredate,v_sal;
                       EXIT WHEN emp_cursor%NOTFOUND;
                       DBMS_OUTPUT.PUT_LINE (v_ename || '    ' ||  v_job || '   '  ||  v_hiredate || '   ' ||  v_sal);
                END LOOP;
                IF emp_cursor%ISOPEN THEN
                       CLOSE emp_cursor;
                END IF;
               DBMS_OUTPUT.PUT_LINE(v_line);
         END LOOP;
        IF emp_cursor%ISOPEN THEN
               CLOSE emp_cursor;
         END IF;
        CLOSE dept_cursor;
END;
```

Output:

	(terminal output below)

```
SQL> @ F:\CSPIT-CE\SEM-4\DBMS\pct_12_3.sql
 44  /
Department Number : 10  Department Name : Administration


Department Number : 20  Department Name : Marketing


Department Number : 30  Department Name : Purchasing

Raphaely    100   07-DEC-02   14000
Khoo    114   18-MAY-03   6200
Baida    114   24-DEC-05   5900
Tobias    114   24-JUL-05   5800
Himuro    114   15-NOV-06   5800
Colmenares    114   10-AUG-07   5500

Department Number : 40  Department Name : Human Resources


Department Number : 50  Department Name : Shipping

Weiss    100   18-JUL-04   11100

Department Number : 60  Department Name : IT

Hunold    102   03-JAN-06   12000
Ernst    103   21-MAY-07   9000
Austin    103   25-JUN-05   7800
Pataballa    103   05-FEB-06   7800
Lorentz    103   07-FEB-07   7200

Department Number : 70  Department Name : Public Relations


Department Number : 80  Department Name : Sales


Department Number : 90  Department Name : Executive

King       17-JUN-03   21900
Kochhar    100   21-SEP-05   20000
De Haan    100   13-JAN-01   20000


PL/SQL procedure successfully completed.
```

Practical – 13

13. **To perform basic PL/SQL blocks**

Q-1 Write a query to create a view for those employees belongs to the location New York.

A

```
SQL> create view employee_newyork as select * from employee where location = 'New York';

View created.

SQL> select * from employee_newyork;

    EMP_NO EMP_NAME                          EMP_SAL   EMP_COMM    DEPT_NO L_NAME                         DEPT_NAME                      JOB_ID
       LOCATION        MANAGER_ID HIREDATE
---------- ------------------------------ ---------- ---------- ---------- ------------------------------ ------------------------------ ---------
------ --------------- ---------- ---------
       107 Anamika                              2975         30         15 Jha                            Artificial Intelligence        it_prog
       New York                15 15-JUL-97
```

Q-2 Write a query to create a view for all employee with columns emp_id, emp_name, and job_id.

A

```
SQL> create view employee_details as select emp_no, emp_name, job_id from employee;

View created.

SQL> select * from employee_details;

    EMP_NO EMP_NAME                       JOB_ID
---------- ------------------------------ ---------------
       101 Smith                          fi_mgr
       102 Snehal                         lec
       103 Adama                          mk_mgr
       104 Aman                           comp_op
       105 Anita                          comp_op
       106 Sneha                          fi_acc
       107 Anamika                        it_prog

7 rows selected.
```

Q-3 Write a query to find the salesmen(it_prog) of the location New York who having salary more than 2000.

A

```
SQL> create or replace view itprog_newyork as select emp_no, emp_name, job_id, emp_sal from employee where location = 'New York' and job_id='it_prog' and emp
_sal > 2000;

View created.

SQL> select * from itprog_newyork;

    EMP_NO EMP_NAME                       JOB_ID             EMP_SAL
---------- ------------------------------ --------------- ----------
       107 Anamika                        it_prog               2975
```

Q-4	Write a query to create a view to getting a count of how many employees we have at each department.

A

```
SQL> create view emp_count as
  2  select dept_no, count(emp_no) as employee_count
  3  from employee
  4  group by dept_no;

View created.

SQL> select * from emp_count;

   DEPT_NO EMPLOYEE_COUNT
---------- --------------
         1              1
        25              1
        20              1
        12              1
        10              2
        15              1

6 rows selected.
```

<table>
<tr><td>Prac 14</td><td>To perfrom the concept of function and procedure</td></tr>
<tr><td>Q-1</td><td>To update the salary of employee specified by empid. If record exist then update the salary otherwise display appropriate

message. Write a function as well as procedure for updating salary</td></tr>
<tr><td>A</td><td>CREATE OR REPLACE PROCEDURE update_salary_procedure(
p_empid IN NUMBER,
p_new_salary IN NUMBER
)
IS
v_count NUMBER;
BEGIN
SELECT COUNT(*) INTO v_count FROM employees WHERE employee_id = p_empid;

IF v_count > 0 THEN
UPDATE employees
SET salary = p_new_salary
WHERE employee_id = p_empid;

COMMIT;
DBMS_OUTPUT.PUT_LINE('Salary Updated Successfully.');
ELSE
DBMS_OUTPUT.PUT_LINE('Employee Not Found.');
END IF;
END update_salary_procedure;</td></tr>
</table>

/

```
SQL> set serveroutput on;
SQL> ed Procedure

SQL> @ Procedure

Procedure created.

SQL> BEGIN
  2      update_salary_procedure(101, 7000);
  3  END;
  4  /
Salary Updated Successfully.

PL/SQL procedure successfully completed.
```

	Practical 15
12.	**To perform the concept of exception handler**
Q-1	Write a PL/SQL block that will accept the employee code, amount and operation. Based on specified operation amount is added or deducted from salary of said employee. Use user defined exception handler for handling the exception.
A	DECLARE v_emp_no employee.emp_no%TYPE := &emp_no; v_amount NUMBER := &amount; v_operation VARCHAR2(1) := UPPER('&operation'); v_old_salary employee.emp_sal%TYPE; v_new_salary employee.emp_sal%TYPE; invalid_operation EXCEPTION; PRAGMA EXCEPTION_INIT(invalid_operation, -20001); BEGIN SELECT emp_sal INTO v_old_salary FROM employee WHERE emp_no = v_emp_no; IF v_operation = '+' THEN v_new_salary := v_old_salary + v_amount; ELSIF v_operation = '-' THEN v_new_salary := v_old_salary - v_amount; ELSE RAISE_APPLICATION_ERROR(-20001, 'Invalid operation specified. Please provide either "+" or "-" as operation.'); END IF;

```
UPDATE employee SET emp_sal = v_new_salary WHERE
emp_no = v_emp_no; COMMIT;

DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');

EXCEPTION

WHEN invalid_operation THEN

DBMS_OUTPUT.PUT_LINE('Error: Invalid operation specified.');

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Error: Employee with code ' || v_emp_no || ' not found.');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);    END;
```

```
SQL> @ exception.sql;
Enter value for emp_no: 101
old   2:     v_emp_no employee.emp_no%TYPE := &emp_no;
new   2:     v_emp_no employee.emp_no%TYPE := 101;
Enter value for amount: 500
old   3:     v_amount NUMBER := &amount;
new   3:     v_amount NUMBER := 500;
Enter value for operation: *
old   4:     v_operation VARCHAR2(1) := UPPER('&operation');
new   4:     v_operation VARCHAR2(1) := UPPER('*');

PL/SQL procedure successfully completed.
```

Prac 16	**To perform the concept of trigger**
Q-1	Write a PL/SQL block to update the salary where deptno is 10. Generate trigger that will store the original record in another table before updation take place.
A	CREATE OR REPLACE TRIGGER trg_backup_before_update BEFORE UPDATE ON employees FOR EACH ROW BEGIN -- Ensure the backup is taken only for department_id = 50 IF :OLD.department_id = 50 THEN INSERT INTO emp_backup (employee_id, first_name, last_name, email, phone_number, hire_date, job_id, salary, commission_pct, manager_id, department_id) VALUES (:OLD.employee_id, :OLD.first_name, :OLD.last_name, :OLD.email, :OLD.phone_number, :OLD.hire_date, :OLD.job_id, :OLD.salary, :OLD.commission_pct, :OLD.manager_id, :OLD.department_id); END IF; END; /

```
SQL> ed trigger

SQL> @ trigger

Trigger created.

SQL> update employees SET salary=salary+5000 where department_id=50;

45 rows updated.
```

Prac 17	**To perform the concept of package**
Q-1	Create a package specification and package body called EMP_PACK that contains your NEW_EMP procedure as a public construct, and your VALID_DEPTID function as a private construct. (You can save the specification and body into separate files.)
A	-- emp_pack_spec.sql CREATE OR REPLACE PACKAGE EMP_PACK AS PROCEDURE NEW_EMP(p_empno employees.employee_id%TYPE, p_ename employees.last_name%TYPE, p_job employees.job_id%TYPE, p_mgr employees.manager_id%TYPE, p_hiredate employees.hire_date%TYPE, p_sal employees.salary%TYPE, p_comm employees.commission_pct%TYPE, p_deptno employees.department_id%TYPE, p_email employees.email%TYPE); END EMP_PACK; / -- emp_pack_body.sql CREATE OR REPLACE PACKAGE BODY EMP_PACK AS

	FUNCTION VALID_DEPTID(p_deptno employees.department_id%TYPE) RETURN BOOLEAN IS v_count NUMBER; BEGIN SELECT COUNT(*) INTO v_count FROM departments WHERE department_id = p_deptno; RETURN (v_count > 0); END VALID_DEPTID; PROCEDURE NEW_EMP(p_empno employees.employee_id%TYPE, p_ename employees.last_name%TYPE, p_job employees.job_id%TYPE, p_mgr employees.manager_id%TYPE, p_hiredate employees.hire_date%TYPE, p_sal employees.salary%TYPE, p_comm employees.commission_pct%TYPE, p_deptno employees.department_id%TYPE, p_email employees.email%TYPE) IS e_invalid_dept EXCEPTION; BEGIN IF NOT VALID_DEPTID(p_deptno) THEN RAISE e_invalid_dept; END IF;

	INSERT INTO employees (employee_id, last_name, job_id, manager_id, hire_date, salary, commission_pct, department_id, email) VALUES (p_empno, p_ename, p_job, p_mgr, p_hiredate, p_sal, p_comm, p_deptno, p_email); COMMIT; DBMS_OUTPUT.PUT_LINE('Employee ' || p_ename || ' inserted successfully.'); EXCEPTION WHEN e_invalid_dept THEN DBMS_OUTPUT.PUT_LINE('Error: Invalid department ID ' || p_deptno || '.'); WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM); ROLLBACK; END NEW_EMP; END EMP_PACK; / -- execute_emp_pack.sql SET SERVEROUTPUT ON; BEGIN EMP_PACK.NEW_EMP(300, 'TestEmployee', 'IT_PROG', 103, SYSDATE, 6000, NULL, 40, 'test@example.com'); END; / BEGIN EMP_PACK.NEW_EMP(301, 'ValidEmployee', 'SA_REP', 145, SYSDATE, 7000, 0.2, 80, 'valid@example.com');

END;

/

```
SQL> -- EMP_PACK Specification (emp_pack_spec.sql)
SQL>
SQL> CREATE OR REPLACE PACKAGE EMP_PACK AS
  2    PROCEDURE NEW_EMP(
  3      p_empno    employees.employee_id%TYPE,
  4      p_ename    employees.last_name%TYPE,
  5      p_job      employees.job_id%TYPE,
  6      p_mgr      employees.manager_id%TYPE,
  7      p_hiredate employees.hire_date%TYPE,
  8      p_sal      employees.salary%TYPE,
  9      p_comm     employees.commission_pct%TYPE,
 10      p_deptno   employees.department_id%TYPE
 11    );
 12  END EMP_PACK;
 13  /

Package created.
```

Q-2

Invoke the NEW_EMP procedure, using 40 as a department number. Because the dept_no 40 does not exist in the DEPT table, you should get an error message as specified in the exception handler of your procedure.

Invoke the NEW_EMP procedure, using an existing department ID 80.

A

```
SQL> BEGIN
  2    EMP_PACK.NEW_EMP(300, 'TestEmployee', 'IT_PROG', 103, SYSDATE, 6000, NULL, 40, 'test@example.com');
  3  END;
  4  /
Employee TestEmployee inserted successfully.

PL/SQL procedure successfully completed.

SQL>
SQL> BEGIN
  2    EMP_PACK.NEW_EMP(301, 'ValidEmployee', 'SA_REP', 145, SYSDATE, 7000, 0.2, 80, 'valid@example.com');
  3  END;
  4  /
Employee ValidEmployee inserted successfully.

PL/SQL procedure successfully completed.
```